

CSD319-DESIGN AND ANALYSIS OF ALGORITHM

Analysis of Parallel algorithms for Graph traversal

REFERENCE PAPER:

PAPER : [Direction-Optimizing Breadth-First Search](#)

**TANIRIKA R (2310110323) . TUHINA AGARWAL (2310110331) . AARIYA JANA (2310110377)
SHRI SUDHAN VIJAYARAJ (2310110293) . SHANMUGANATHAN RAMAKRISHNAN (2310110273)**

Problem Background

Graph traversal is an important operation used in computing, shortest path algorithms, finding connectivity, social network analysis, recommendation systems, and data mining. However, modern real-world graphs such as social networks and web graphs are huge, uneven, and show properties like low diameter (that is everyone is connected) and scale-free degree (few nodes are super connected hubs, while other nodes only have few links) distributions, making traversals very costly on traditional sequential algorithms. Conventional parallel BFS suffers from poor locality- could be scattered on memory, multiple threads trying to access the same data, and significant redundant work, because millions of edges must be simultaneously searched at every frontier expansion. According to the paper, the bottleneck is not computation but memory-bound edge checking, with most neighbor checks turning out to be unnecessary. This inefficiency motivates the need for smarter, direction-aware parallel algorithms that reduce work, improve memory behavior, and effectively scale across multicore architectures.

Algorithmic Approach

Parallel BFS processes all nodes in the frontier together by dividing them across threads, each thread explores neighboring nodes in parallel. On large low diameter and scale free graphs this approach becomes less efficient because these threads repeatedly check edges which are already visited thereby creating large amounts of redundant work. The dispersed memory layout of adjacency lists leads to poor locality which causes frequent cache misses, while multiple threads trying to check the same neighboring nodes which causes heavy contention. These issues make conventional parallel BFS memory bound and limits the scalability on modern multicore systems.

```
function breadth-first-search(vertices, source)
    frontier  $\leftarrow$  {source}
    next  $\leftarrow$  {}
    parents  $\leftarrow$  [-1,-1,...-1]
    while frontier  $\neq$  {} do
        top-down-step(vertices, frontier, next, parents)
        frontier  $\leftarrow$  next
        next  $\leftarrow$  {}
    end while
    return tree
```

Fig. 1. Conventional BFS Algorithm

The Direction-Optimizing BFS proposes a hybrid parallel algorithm that changes between top-down and bottom-up traversal based on frontier size. When the frontier is small, a parallel top-down search is preferred and more efficient. Once the frontier size becomes large, threads change to a parallel bottom-up approach, where each thread processes a sub part of the unvisited nodes and checks whether *any* of their neighbors are present in the frontier. This early stop behavior shrinks edge checking and eliminates contention because each node writes only to its own parent slot, and improves memory behavior. A

simple heuristic decides when to change directions, allowing the hybrid algorithm to maintain correctness while achieving increased speedups, often $3\times$ – $8\times$ faster than conventional parallel BFS on real world graphs making it the most efficient approach for parallel graph traversal.

Nf = Frontier size , mf = total edges , mu = upper bound on edges unvisited nodes may scan

```

function top-down-step(vertices, frontier, next, parents)
  for v ∈ frontier do
    for n ∈ neighbors[v] do
      if parents[n] = -1 then
        parents[n] ← v
        next ← next ∪ {n}
      end if
    end for
  end for

```

Fig. 2. Single Step of Top-Down Approach

```

function bottom-up-step(vertices, frontier, next, parents)
  for v ∈ vertices do
    if parents[v] = -1 then
      for n ∈ neighbors[v] do
        if n ∈ frontier then
          parents[v] ← n
          next ← next ∪ {v}
          break
        end if
      end for
    end if
  end for

```

Fig. 5. Single Step of Bottom-Up Approach

The hybrid BFS uses two thresholds values α and β to check when to change directions. It calculates mf and mu. The algorithm changes to the bottom-up approach when $mf > mu/\alpha$, $\alpha = 14$ was tuned experimentally to initiate the switch exactly when frontier growth makes top-down redundant. nf is compared to the total no. of nodes n and the algorithm changes back to top-down approach when $nf < n/\beta$; $\beta = 24$ was selected because bottom-up becomes less efficient once few unvisited nodes are left. These α and β values were chosen so that the heuristic performs within 10–25% of an oracle's optimal switching points.

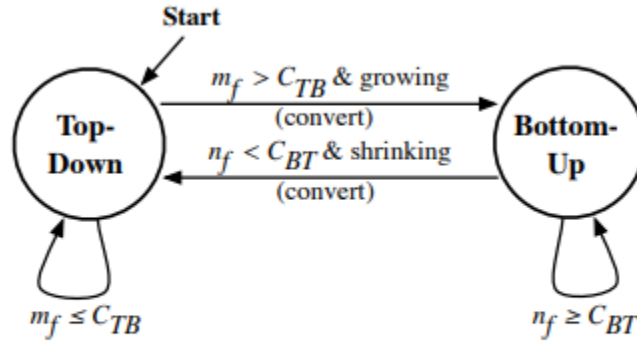


Fig. 7. Control algorithm for hybrid algorithm. (convert) indicates the frontier must be converted from a queue to a bitmap or vice versa between the steps. Growing and shrinking refer to the frontier size, and although they are typically redundant, their inclusion yields a speedup of about 10%.

Experimental Implementation and Results

To evaluate the parallel BFS implementation, we compiled the source using g++ with OpenMP enabled so that multiple threads could be used during traversal. After being successfully compiled, a BFS executable was generated, which we ran by providing a graph file as input. The program executed BFS on the supplied graph and generated the total execution time. We ran the program with different thread counts (1, 2, 4, 8, and 16 threads) to observe the speed up of the parallel algorithm and recorded the outputs for analysis.

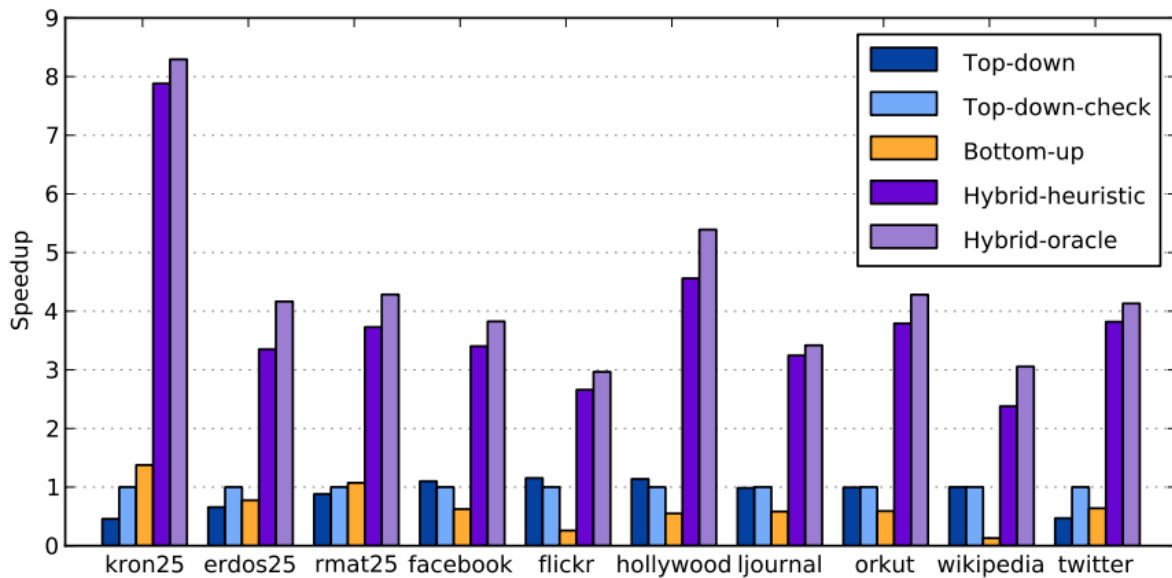


Fig. 10. Speedups on the 16-core machine relative to *Top-down-check*.

Increasing the thread count reduces efficiency because BFS is a memory-bound and irregular algorithm. As more threads were introduced, they started competing for the same memory, causing halts instead of speeding up the work. At the same time, the amount of useful parallel work in BFS often reduces at each level as many levels have very small frontiers so more threads simply sit idle or add synchronization and atomic-update (only one thread is allowed to access the data) overhead. Due to memory contention, load imbalance, and increased coordination cost means that beyond a limit, more no. of threads actually decreases performance, reducing efficiency.

FULL THREAD SCALING RESULTS

Threads	Execution Time (s)	Speedup	Efficiency
1	0.00400	1.00x	100%
2	0.00300	1.33x	67%
4	0.00000	∞	-
8	0.00000	∞	-
16	0.00400	1.00x	6%

Research Gaps and Future Directions

1. Performance on High-Diameter Graphs: The Direction-Optimizing approach performs well for low-diameter social networks but provides no benefit for high-diameter graphs, such as physical meshes, where the search frontier remains small. One promising direction for future systems is to use a "meta-hybrid" strategy: before execution, the system analyzes the graph structure, and if it detects a high-diameter graph, it uses the traditional graph partitioning approach; if it detects a low-diameter graph, then the Direction-Optimizing approach should be used.

2. Algorithmic Generalization: The methods described in the paper are currently made for BFS, and its specific parent-claiming method. Expanding this "push/pull" logic to other graph types would be the next step. Frameworks, such as Ligra, have shown that the ability to switch between sparse and dense traversal modes can accelerate a broad class of algorithms, beyond BFS, including Betweenness Centrality and Connected

[Ref] [Ligra: A Lightweight Graph Processing Framework for Shared Memory](#)

3. Scalability and Distributed Systems: The current algorithm is targeted at shared-memory architectures. An extension to distributed clusters is hard since the relative network communication cost often offsets the parallelism advantage for irregular problems such as BFS. Thus, for future work, the emphasis should be on "strong scaling" of large, single-node shared-memory systems to avoid network bottlenecks rather than attempting to scale across distributed clusters.

4. Integration with Low-Level Optimizations: The proposed hybrid approach focuses on algorithmic efficiency—specifically reducing the number of edge checks—and is mostly independent of hardware-level tuning. Future implementations could combine this direction-switching logic with lower-level optimizations, such as the memory layout reordering and dynamic load balancing proposed by Chhugani et al. Integrating the algorithmic reduction of work with these hardware-centric optimizations may lead to even higher performance on shared-memory systems.

[Ref] Jatin Chhugani et al. Fast and efficient graph traversal algorithm for CPUs: Maximizing single-node efficiency. International Parallel and Distributed Processing Symposium, 2012

5. Adaptive Tuning: Currently, the decision to switch traversal directions depends on fixed and pretuned parameters that might not be optimal for every type of graph or hardware configuration. Future systems need to shift toward adaptive tuning mechanisms, which would automatically set the switching thresholds at runtime depending on the particular behavior the graph exhibits at runtime, without manual calibration.